Portfolio

Full-Stack AI Engineer · kc-bibek.com.np

Bibek K C

SELECTED PROJECTS

KhururuAI — Work in Flow (Web platform · 2026)
Charak — Medical Research Assistant (Web app + MCP server · 2026)
Bintech International (Website + CRM · 2026)
AirConnect — Proximity-Based Social Discovery (Mobile app · 2025)
B-Assist (Better-Assistant) (Desktop app, Windows/macOS · 2024)
ReNameify — Content-Aware PDF Batch Renamer (Desktop app, Windows · 2024)

RESEARCH AND APPLIED ML PROJECTS (2023–2024)

FastRAG — Self-Hosted Document RAG Backend (2024)
Research on Probabilistic Multiview Object-Pose Estimation (2023)
6D Pose Estimation and Scene Graph Generation (2023)
MBTI Prediction (2023)
ESL Fusion — Junction Asia Hackathon (2023)
B-bot (Personal Chatbot) (2023)

EARLIER PROJECTS AND COURSEWORK (≤2022)

Paraphrase Generation (QCPG Replication)
AI-empowered F1/10 Autonomous Racing Car
Crypto Price Predictor
Movie Recommendation (Netflix Prize style)
Lecture Summarizer
Pintos
Smart Home
Data Cleanup
Data Visualization Task

SELECTED PROJECTS

Products, designed and built end to end.

KhururuAI — Work in Flow (Web platform · 2026)

The point of this project is access. Capable AI tools price out exactly the people who would gain most from them, while every major provider — Groq, Cerebras, NVIDIA, Google, Mistral — gives away a free tier that goes unclaimed because claiming it means holding an API key. KhururuAI pools that free capacity into a platform anyone can just open and use: no payment, no API key, and no account at all for the first five turns. It runs today on my own pooled keys, roughly **2.3B tokens** of budget; the version I'm working toward provisions each user's *own* free tier at sign-in, so the ceiling scales with users instead of being divided among them.

A user writes what they want in plain language, an LLM router picks one of six specialists, and that specialist returns the finished thing — a researched answer with live citations, a slide deck, a PDF, working code, an image, or ordinary conversation. Uploaded documents are parsed, embedded and retrieved with page-accurate citations. Turns stream over SSE as typed events, so the interface names the specialist at ~1–2s and the model that is writing before the first token exists.

Staying free without falling over is the engineering problem. One OpenAI-compatible gateway fronts every provider, collapsing three vendor SDKs into a single client where "which provider" becomes "which model id"; agents are pinned to capability *slots* rather than vendors, so a model that runs dry is a one-line swap and an exhausted route falls back on its own. Free access sits behind two ceilings doing different jobs — a guest allowance that deliberately never refills, and a per-IP daily budget that stops the endpoint being farmed. Five Node services plus Redis, Qdrant and MinIO behind a React SPA, reproducible with one Docker Compose command, with the decisions recorded as ADRs.

> **Tools:** Node.js, Express, LangGraph.js, React, TypeScript, MongoDB, Redis, Qdrant, MinIO/S3, Docker Compose, nginx, Firebase Auth, Tavily
>
> **ML/Algorithms:** LLM intent routing across an agent roster; multi-provider model routing with exhaustion fallback; document RAG — page-aware chunking, Gemini embeddings, score-floored retrieval under a hard context budget, validated citations
>
> **Links:** khururu.com

Charak — Medical Research Assistant (Web app + MCP server · 2026)

Charak answers natural-language clinical questions from live **PubMed** literature and **ClinicalTrials.gov v2** records, and charts the answer when the question is really about a distribution. The hard part was the public API, not the model. I verified every capability with live `curl` first, which caught the wrong assumptions: the obvious filters (`filter.phase` , `filter.country`) don't exist — real narrowing is Essie, `AREA[Phase]PHASE3` — and the one aggregation endpoint can't be scoped to a query and returns no trial IDs, so it can't back citations either. With no server-side aggregation, counts are resolved in tiers: exact from the full corpus, exact via a cheap per-bucket `countTotal` query when it's too large to fetch, sampled-but-flagged only for

unbounded fields. Because aggregation walks records instead of asking for a summary, every bar carries the trial IDs that built it.

For deterministic work, a tool-calling agent beats code-gen or one big prompt. The LLM owns language and control flow only, orchestrating typed Python tools, and never sees or writes a number — state stays server-side and it passes only opaque dataset handles, so it cannot emit a wrong figure because it is never handed one. Chart-type selection is deterministic rules; the "no matching trials" reply is code-gated rather than left to judgment. That costs more round-trips, but each is a short tool call instead of a wall of records pushed through context — cheaper per query and structurally correct. The backend renders nothing: it returns a small flat JSON spec that Vega-Lite draws on the client, thin enough that one response feeds the chat UI, the `POST /visualize` API and a standalone **MCP server** unchanged.

> **Tools:** Python, FastAPI, Pydantic v2, MCP (Model Context Protocol), Vega-Lite, vis-network, Server-Sent Events, Docker Compose, Caddy, pytest, uv
>
> **ML/Algorithms:** Tool-calling agent loop with server-side state handles (no numeric values in context); three-tier exact-count resolution over a non-aggregating API; deterministic data-shape → chart-type selection; entity-normalized co-occurrence network construction; provider-swappable LLM layer (OpenAI / Anthropic)
>
> **Links:** charak.khururu.com · GitHub

Bintech International (Website + CRM · 2026)

Built to survive being handed over. Bintech is a B2B hardware supplier in Nepal; I shipped the whole thing solo — public site, staff CRM, AI assistant, and the server it runs on. Nobody there writes code, so the design is only correct if a salesperson can add a product, publish it and answer a customer without me. That put the catalogue in **Payload CMS 3** over Postgres with real access control (anonymous reads constrained *by query* to published, staff see drafts) and admin copy written for the person typing into it. Inquiries invert the usual permissions — the public creates but can't read, staff read but the submitted fields are read-only, so the only editable thing is the New → Contacted → Closed status. Failure modes took most of the attention: every catalogue query falls back to a static array, so the image builds with no database and a Postgres outage degrades to the seed catalogue instead of a blank page, and inquiry emails fire from a post-persist hook — an SMTP outage costs a notification, never a lead.

The assistant is deliberately the boring kind. Each request flattens the live catalogue into one compact line per product behind a 60s cache and pins it into a guardrailed system prompt that forbids inventing prices and hard-codes the only two legal URL shapes (LLMs cheerfully guess `/products/monitors`, which 404s). Answers stream back as plain-text deltas; the provider key never reaches the browser, requests are per-IP rate-limited and length-capped, and each completion logs one structured line so a truncated answer is diagnosable from logs rather than guessed at. **Grounding beats fine-tuning here** — a price edit in the CRM is live in the bot's answers within a cache window, and it cannot quote a number that isn't in the database. It self-hosts on one VPS via Docker Compose behind Caddy, deployed by a GitHub Actions job holding no repo permissions against an SSH key locked server-side to a forced command. Domain, DNS and company mailboxes are mine too.

> **Tools:** Next.js 16 (App Router), React 19, TypeScript, Tailwind CSS v4, Payload CMS 3, PostgreSQL 17, Drizzle migrations, Nodemailer/SMTP, sharp, Motion, Docker Compose, Caddy, GitHub Actions
>
> **ML/Algorithms:** Catalogue-grounded LLM assistant — live DB → compact token-efficient context behind a TTL cache, guardrailed system prompt (no invented prices/specs, closed link vocabulary, out-of-scope hand-off), SSE → plain-text delta re-emission, per-IP rate limiting and per-completion telemetry
>
> **Links:** bintechint.com

AirConnect — Proximity-Based Social Discovery (Mobile app · 2025)

The idea was mine. You are on a subway and the person across from you is reading a book you love, and there is no socially cheap way to say so. AirConnect was the icebreaker — ping the one person in your line of sight, so the first move costs nothing.

The bet was **Ultra-Wideband: the only widely shipped radio that gives angle of arrival, not just range** — and "that person, over there" is an angle problem. Bluetooth knows someone is nearby, never which one. The UWB SDKs proved too unstable to build on, and without direction the premise was gone. We pivoted to GPS-based nearby discovery, which put us in dating-app territory — where **Apple now rejects new entrants that aren't meaningfully differentiated**. So we stopped. Two people, six weeks: a working real-time app, and a clear answer to why the idea doesn't survive current hardware and platform policy.

An Expo/React Native client and a Node + Socket.IO backend — nearby discovery, matching, chat, and live location sharing on a map, with Redis as the primary datastore. Binh built the foundation and owned the map/GPS and chat features. **I owned the backend and infrastructure**: Docker/Compose, then **restructuring a 2,500-line single-file Express server into controllers, services and routes** with a PostgreSQL migration path alongside Redis — plus the welcome/signup/onboarding flow and a polish pass over Binh's map (marker-flicker fix, `DirectionMarker` and compass heading).

> **Tools:** Expo, React Native, TypeScript, Node.js, Express, Socket.IO, Redis, PostgreSQL, Docker Compose, cloudflared

B-Assist (Better-Assistant) (Desktop app, Windows/macOS · 2024)

Better-Assistant is a fun project, designed and developed to gain experience in building end-to-end, user-centric applications. The final product is a one-click executable that can be easily used by typical PC (win32/macOS) users. It integrates AI features into commonly used low-spec PCs, lowering the entry barrier for new contributors. Currently, it functions as a basic writing assistant, enhancing the writing experience by incorporating generative models into your preferred text editors. Future updates will extend its capabilities to include input audio processing (using Whisper) and screen understanding (using vision models).

> **Tools:** ElectronJS, ReactJS, robotJS, Ollama
>
> **Links:** GitHub · Demo

ReNameify — Content-Aware PDF Batch Renamer

(Desktop app, Windows · 2024)

Written for one real user, not a spec. My sister was renaming exported PDFs by hand — opening each one, reading the header, retyping it — so I sat with her, watched the actual workflow, and shipped a one-click Windows installer that does it in a folder-at-a-time pass. It extracts each PDF's text, then scores lines with a header-detection heuristic that rejects the things that look like titles but aren't: page numbers (`page 12`, `- 12 -`), sentences ending in a full stop, lines that are mostly punctuation, and anything outside a sane length band — Korean-aware, since half her documents were in Korean. **Trust mattered more than accuracy here:** nothing is renamed in place. Originals stay untouched and copies land in a `renamed_files/` subfolder, a PDF that fails to parse (scanned pages, no text layer) keeps its original name instead of erroring the batch, and a first-run guided tour walks a non-technical user through the three buttons. It survived contact with its one user, which is the only test that counted.

> **Tools:** ElectronJS, Node.js, pdf-parse, fs-extra, Shepherd.js, Winston, electron-builder (NSIS one-click installer)
>
> **ML/Algorithms:** Rule-based header extraction from first-page text — length banding, terminal-punctuation and page-number rejection, special-character-ratio filtering with a Hangul-safe character class, document-specific anchor matching, and filename sanitization with graceful fallback to the original name
>
> **Links:** GitHub

RESEARCH AND APPLIED ML PROJECTS (2023–2024)

Where the model or the method was the deliverable.

FastRAG — Self-Hosted Document RAG Backend (2024)

No API key, no data leaving the machine. FastRAG is a document Q&A backend that comes up with a single `docker compose up`: a FastAPI parse/chat service, Qdrant for vectors, a CUDA llama.cpp server running a quantized Llama-3.2-3B, and a Gradio demo UI. Ingestion offers two paths — a custom PyMuPDF reader with tunable chunk size and stride, plus an optional bridging pass that stitches the tail of one page to the head of the next so an answer split across a page break is still retrievable as one chunk; or layout-aware Markdown chunking for documents where structure carries the meaning. Retrieval is bge-m3 dense search with a **score floor**, and the prompt is written to refuse rather than improvise when the retrieved references don't cover the question. Follow-ups can be rewritten into standalone queries against conversation history kept in Qdrant, and answers stream token-by-token from llama.cpp through FastAPI to the UI, with per-response latency logged and shown to the user.

> **Tools:** Python, FastAPI, Qdrant, llama.cpp (CUDA server), LlamaIndex, PyMuPDF/pymupdf4llm, LangChain text-splitters, Gradio, Docker Compose, Poetry, pre-commit
>
> **ML/Algorithms:** BAAI/bge-m3 dense embeddings with cosine search and a score threshold; on-disk vectors and memmap-tuned Qdrant collections for low-RAM hosts; page-boundary overlap chunking

> vs. layout-aware Markdown chunking; history-conditioned query rewriting; grounded-answer prompt with an explicit refusal path
>
> **Links:** GitHub

Research on Probabilistic Multiview Object-Pose Estimation (2023)

We proposed a novel probabilistic multiview object-pose estimation framework that associates multiple estimation results with scene graphs, combines pose distributions from single-view based estimators, and constructs a unified scene graph by predicting a unified pose distribution per object. We evaluated our method with table-top object-pose estimation scenarios and showed the proposed approach outperforms three baseline methods in terms of position and orientation accuracies.

> **Role:** Neural architecture research for pose estimation and scene graph generation, followed by training and combined efficient inference. My teammate handled environment setup, simulation dataset creation, and visualization.
>
> **Links:** Draft paper

6D Pose Estimation and Scene Graph Generation (2023)

I used the Cosypose framework for 6D pose and the Neural-Motif framework for SGG, and performed experiments on custom cup datasets. This was done for a single-camera setup, estimating 6D pose and generating a dynamic scene graph. For 6 cups, with 6D pose estimation plus relationship and attribute predictions, the combined framework ran at 1.25 frames/s.

> **ML/Algorithms:** Cosypose — CNN with geometric optimization (ICP registration and non-linear optimization via Levenberg-Marquardt); Neural-Motif — Region Proposal Network with bi-directional LSTM
>
> **Links:** Demo

MBTI Prediction (2023)

Submitted to the Qualcomm-KAIST Innovation Awards Hackathon in May 2023. In this work I predicted MBTI, a widely used personality assessment tool, from text data. My proposed model is a hierarchical classifier-based approach with 4 fine-tuned KoBERT classifiers and 4 SVM classifiers, one pair per section of the MBTI test. To improve robustness I employed a data augmentation technique, paraphrasing long answers using google-t5-base. I performed EDA on the question information and extracted meaningful information to create a new set of features from the questions. For fine-tuning KoBERT, a fully connected layer with softmax activation was added at the end, and I adaptively fine-tuned the model by varying the amount of layer-freezing in two steps.

The overall results were satisfactory and I secured **3rd position** in the Phase-2 private leaderboard. For more details on results, other approaches I tried, and future work, see the slides.

> **ML/Algorithms:** KoBERT fine-tuning, SVM, hierarchical classification, paraphrase-based data augmentation
>
> **Links:** Presentation

ESL Fusion — Junction Asia Hackathon (2023)

Our team used the Electronic Shelf Label (ESL) solution provided by SOLUM. Our approach implemented a recommendation system for offline stores, harnessing energy-efficient and cost-effective ESLs that support scalability. We streamlined store stock information for recording and tracking, implemented data visualization and analysis on stock data to extract insights, and applied time series forecasting to project the next day's sales.

> **Role:** Entrepreneur. I participated in brainstorming, assigned roles to team members, and created the presentation slides. Leveraging my developer background, I studied their API manuals and worked out how to send POST requests to display our content on the ESL screen. I also generated QR codes and built a simple web interface with visualizations for customers, accessible on the local network after scanning the QR code.
>
> **Tools:** REST API, qrcode, Dash HTML, Matplotlib, Flask, Figma
>
> **Links:** GitHub

B-bot (Personal Chatbot) (2023)

A personal chatbot that serves as an efficient way of retrieving information about me that is listed in my resume. It provides prompt responses to queries about my skills, education, work experience, and other relevant information without the need to look through the resume itself. The chatbot takes prompts from the user and classifies them against predefined tags, trained on a file of labeled question-answer sets. I used an MLP classifier with three hidden layers and ReLU activations, and a Bag of Words representation to extract relevant keywords and return the appropriate response. I also implemented text preprocessing such as stemming and spell-checking to improve response correctness. The interface was built with Streamlit and hosted on their platform.

> **Tools:** Streamlit-chat, PyTorch, PySpellchecker, NLTK
>
> **ML/Algorithms:** Multi-Layer Perceptron classifier, Bag of Words representation
>
> **Links:** Live chatbot · GitHub

EARLIER PROJECTS AND COURSEWORK (≤2022)

University coursework, team projects and first builds.

Paraphrase Generation (QCPG Replication)

We replicated *Quality Controlled Paraphrase Generation* (Bandel et al., ACL 2022).

> **Role:** Improvement approaches. First, I checked the empirical importance of the Quality Predictor model by comparing QCPG results with and without it. Second, I worked out how to swap t5-base for multilingual mt5-base to train on a Korean dataset. Third, I wrote an end-to-end paraphrasing inference pipeline from scratch.
>
> **Tools:** PyTorch, Matplotlib, transformers, Hugging Face models
>
> **Links:** Replication report · GitHub

AI-empowered F1/10 Autonomous Racing Car

A team project for a course. Since I am more comfortable with Linux, I was in charge of setting up the environment and networking on the Jetson Xavier NX, including installing libraries and ROS packages for the sensor interfaces. Other team members handled hardware — Arduino, LIDAR, IMU, camera, chassis, and wiring.

My main responsibility was perception, specifically object detection. I started with Yolact_ros — instance-segmentation based and more accurate — since we planned to extend 2D detections to 3D for obstacle avoidance. It proved computationally heavy, so we moved obstacle avoidance to LIDAR data instead. Ultimately, for the same computational reasons, I used a simple rule-based classifier involving cropping, masking, and counting red, green, and yellow pixel values, and we used the vision model only for traffic light detection. For localization we used AMCL and Hector Odom; for control, PID. I also helped with algorithm testing and parameter tuning in the real-world environment.

> **Tools:** Yolact_ros, OpenCV, Linux, ROS, NVIDIA SDK, Arduino IDE
>
> **Links:** Final report

Crypto Price Predictor

We used time-series forecasting and attention mechanisms to predict the future price of bitcoin, developing seven forecasting models — from deep neural networks to a Bayesian network-based model — across two tasks: one-day and long-sequence (up to 100 days) forecasting.

> **Role:** I experimented with two deep learning models for the one-day task: one recurrent (bidirectional LSTM) and one convolutional (Conv1D). I also did multivariate forecasting with both.
>
> **Tools:** TensorFlow, Pandas, Matplotlib
>
> **ML/Algorithms:** Conv1D, bi-directional LSTM
>
> **Links:** Project paper · GitHub

Movie Recommendation (Netflix Prize style)

My task was to fill in the `<RATING>` column of an input file containing `<USER ID>`, `<MOVIE ID>`, and `<TIMESTAMP>` entries. To estimate the blank entries in the utility matrix — n users by m movies, with ratings as entries — I used UV-decomposition for dimensionality reduction, approximating the matrix as $R \approx UV^T$

where U is n × k and V is m × k, with k smaller than both n and m. Each user and movie is represented by a k-dimensional vector of latent factors. To train the decomposition I used Alternating Least Squares to minimize RMSE. Before training I performed two-fold normalization, subtracting the user's average rating and then the movie's. Finally I denormalized the utility matrix and used U and V to predict the rating for each user-movie entry.

> **Tools:** NumPy, Python
>
> **ML/Algorithms:** Dimensionality reduction (UV-decomposition), Alternating Least Squares

Lecture Summarizer

> **Role:** Documentation — the Project Planning & Management Document (PPMD), Software Architecture Document (SAD), and System Testing Document. Around 80% of these were written by me. I was not explicitly involved in development; of our team of 4, three handled frontend, backend, and NLP. I helped with two issues: converting text to PDF (formatting broke due to missing EOL characters in the text format), and managing project dependencies with conda plus updating the README so the code worked on Linux, macOS, and Windows.
>
> **Tools:** Django, React.js, rule-based NLP, Git, Notion, Figma
>
> **Links:** Presentation · GitHub

Pintos

The course project for my Operating Systems course, implementing pintos-kaist, a simple operating system framework for x86-64. Forked from the Pintos project at Stanford, run under QEMU. In the first project my task was to solve thread synchronization problems with interrupts using semaphores, locks, and condition variables. In project 2 I implemented a system call handler for I/O interactivity in running user programs. In project 3 I implemented virtual memory management, including page fault handling and swapping, to build an illusion of infinite memory. In the final project I worked on the file system and got the basic file system supporting addition, deletion, and manipulation of files. Overall a great experience working in a large repository.

> **Links:** Project reference

Smart Home

A security tech project detecting intruders and notifying the house owner.

> **Role:** Hardware setup (connecting the sensor to a Raspberry Pi via breadboard) and the Node-RED workflow, connecting each process to a Telegram bot for automated response.
>
> **Tools:** Node-RED
>
> **Links:** Demo video · GitHub

Data Cleanup

> **Role:** Data visualization and webpage design; my teammate handled data pre-processing.
>
> **Tools:** Python, Plotly, Pandas, Flask, Dash, Heroku
>
> **Links:** Live app · Demo video · GitHub

Data Visualization Task

A visualization task given to me by a startup before hiring me as an intern in Winter 2022.

> **Tools:** D3.js, Colab (data preprocessing), Flask, Heroku
>
> **Links:** Live app · GitHub